Listing 7.48 Custom analyzer with additional filters

```
PUT index_with_token_filters
{
  "settings": {
    "analysis": {
      "analyzer": {
        "token_filter_analyzer": {
          "tokenizer": "standard",
          "filter": [ "uppercase","reverse"]
        }
      }
    }
  }
}
```

Defines a custom analyzer

Provides token filters as an array of filters

7.8.1 The stemmer filter

As explained earlier in the chapter, *stemming* reduces the words to their root words (for example, *bark* is the root word for *barking*). Elasticsearch provides an out-of-the-box stemmer that reduces the words to their root form. The next listing shows an example.

Listing 7.49 Using a stemmer as the token filter

```
POST _analyze
{
  "tokenizer": "standard",
  "filter": ["stemmer"],
  "text": "barking is my life"
}
```

When executed, this code produces a list of tokens: “bark”, “is”, “my”, and “life”. The original word “barking” is transformed to “bark”.

7.8.2 The shingle filter

Shingles are word n-grams generated at the token level (unlike n-grams and edge n-grams that emit n-grams at the letter level). For example, the text, “james bond” emits the shingles “james” and “james bond”. Here’s an example of a shingle filter:

```
POST _analyze
{
  "tokenizer": "standard",
  "filter": ["shingle"],
  "text": "java python go"
}
```

The result is `[java, java python, python, python go, go]`. The default behavior of the filter is to emit unigrams and two-word n-grams. We can change this default behavior by creating a custom analyzer with a custom shingle filter.